Software Project Management – IS212

User Stories

Announcements

Announcements – Slack

- Reminder that Slack is our primary communication platform
  - https://is212-2025.slack.com/
- You should be able to log in using Google SSO with your SMU account
- **Email me if not** (especially if you joined in BOSS 2/2A)
- **Please do**:
  - Add your section to your name
  - Upload a photo
  - Take part in #content-sharing and #help-is212

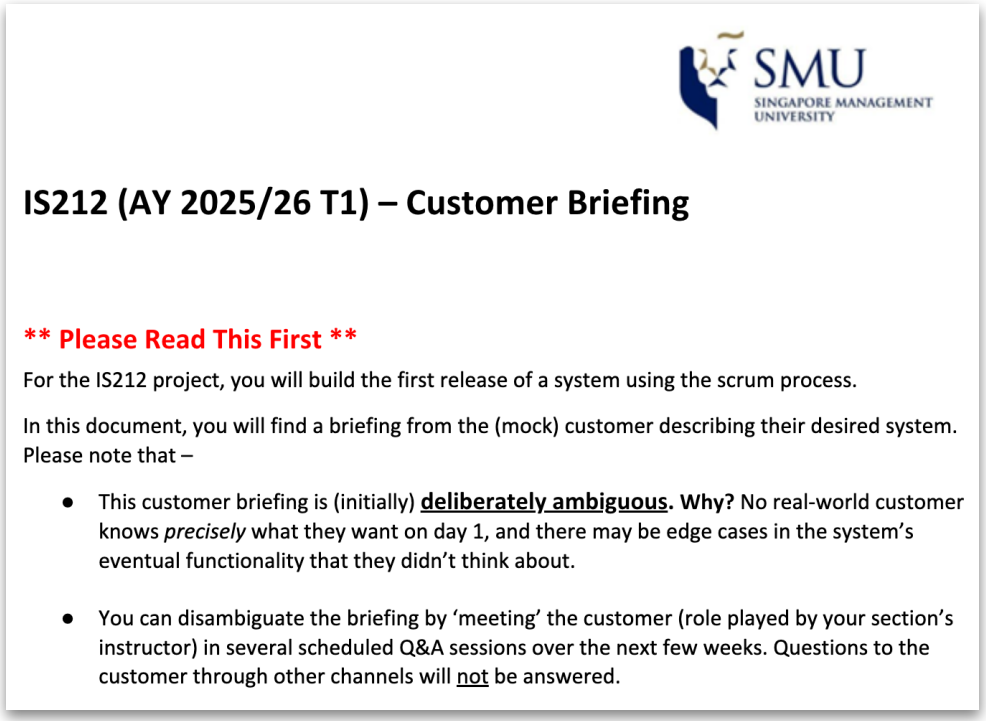


SMU Classification: Restricted

Announcements – Week 3 Quiz

- Our first quiz will take place at the start of your Week 3 class
  - *Starts promptly at the beginning of class!*
  - *No extra time if you arrive to class late*
- Need to be physically present (no make-ups; see Week 1 slides)
- Requires Respondus LockDown Browser
  - *Test your installation using the "dummy quiz" on eLearn*
- Short (~10-15 mins); covers the lessons of Weeks 1–2
  - *MCQ-style questions (no written)*

Announcements – Project Customer Briefing



SMU
SINGAPORE MANAGEMENT UNIVERSITY

IS212 (AY 2025/26 T1) – Customer Briefing

**** Please Read This First ****

For the IS212 project, you will build the first release of a system using the scrum process.

In this document, you will find a briefing from the (mock) customer describing their desired system. Please note that –

- This customer briefing is (initially) **deliberately ambiguous**. **Why?** No real-world customer knows *precisely* what they want on day 1, and there may be edge cases in the system's eventual functionality that they didn't think about.
- You can disambiguate the briefing by 'meeting' the customer (role played by your section's instructor) in several scheduled Q&A sessions over the next few weeks. Questions to the customer through other channels will not be answered.

Available in eLearn now

Start organising yourself into groups

***Project Instructions* –** Mon Wk4 8am

What should you do & not do until Mon Week 4…?

Recap

Week 1 Recap – Software Development Processes

- Traditional software development processes
  - *waterfall, RUP* 

User stories in product backlog can be changed but once it is in the sprint, it cannot be changed. Unfinished user stories after a sprint goes back into the product backlog.

- Rejection of "big upfront" steps – Agile Manifesto (2001)

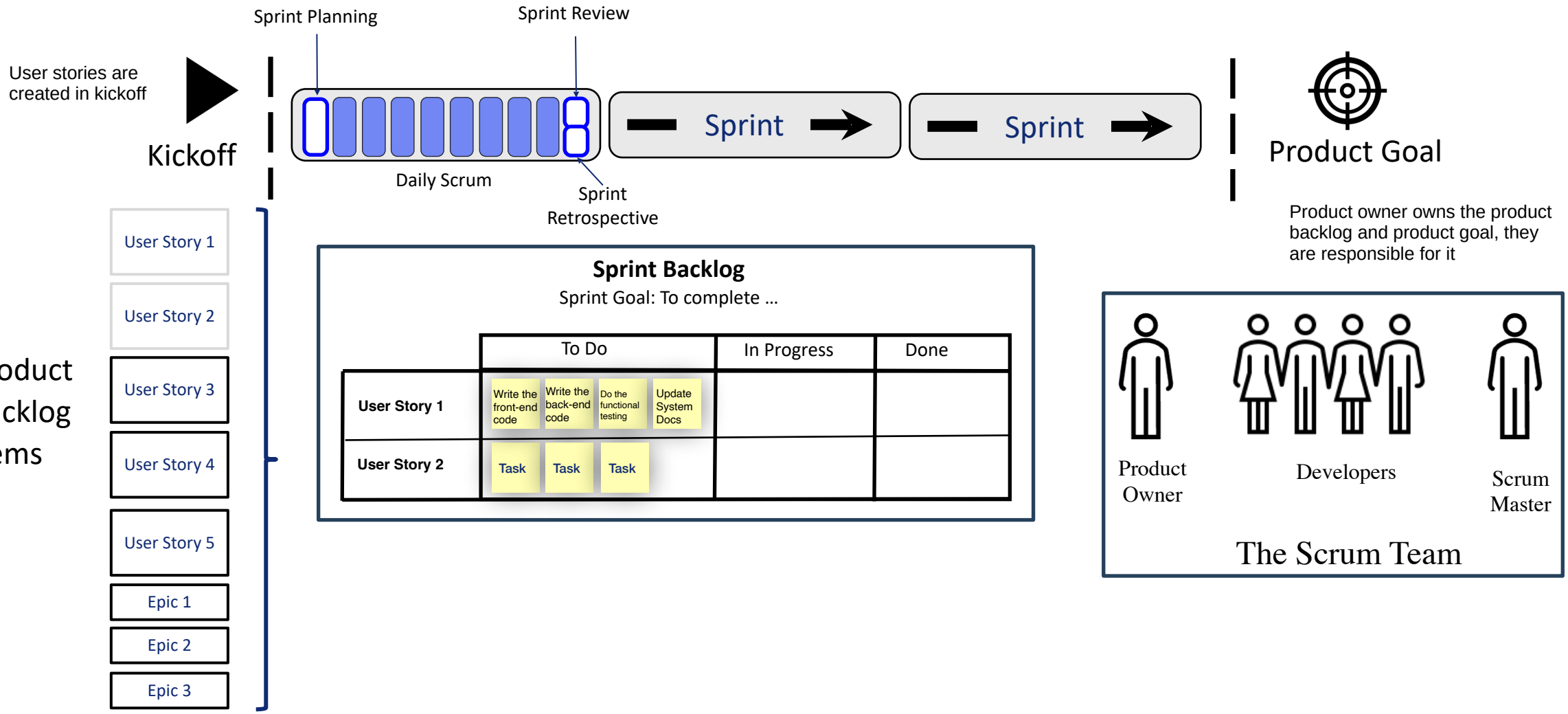
Agile values

1 Redefined roles for developers, managers and customers.
2 No "Big Upfront" steps.
3 Iterative development.
4 Limited, negotiated functionality.
5 Focus on quality, understood as achieved through testing.



- Agile methodologies
  - *Scrum – prescriptive iteration model; frozen sprint goals*
  - *XP – pair programming, test-driven development, frequent prototyping, …*
  - *Kanban – visualise your workflow; minimise waste*

Week 1 Recap – Scrum



SMU Classification: Restricted

User Stories

User Stories

Objectives

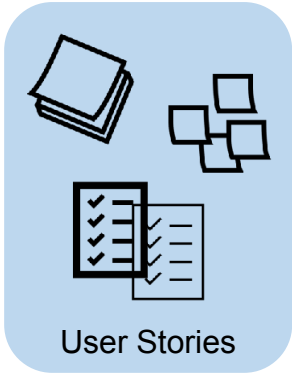
On completing this class, you will be able to:

- Explain the rationale for writing user stories and epics
- Create user story cards to describe the features of a system
- Evaluate the quality of a user story card against the "INVEST" criteria

Outline

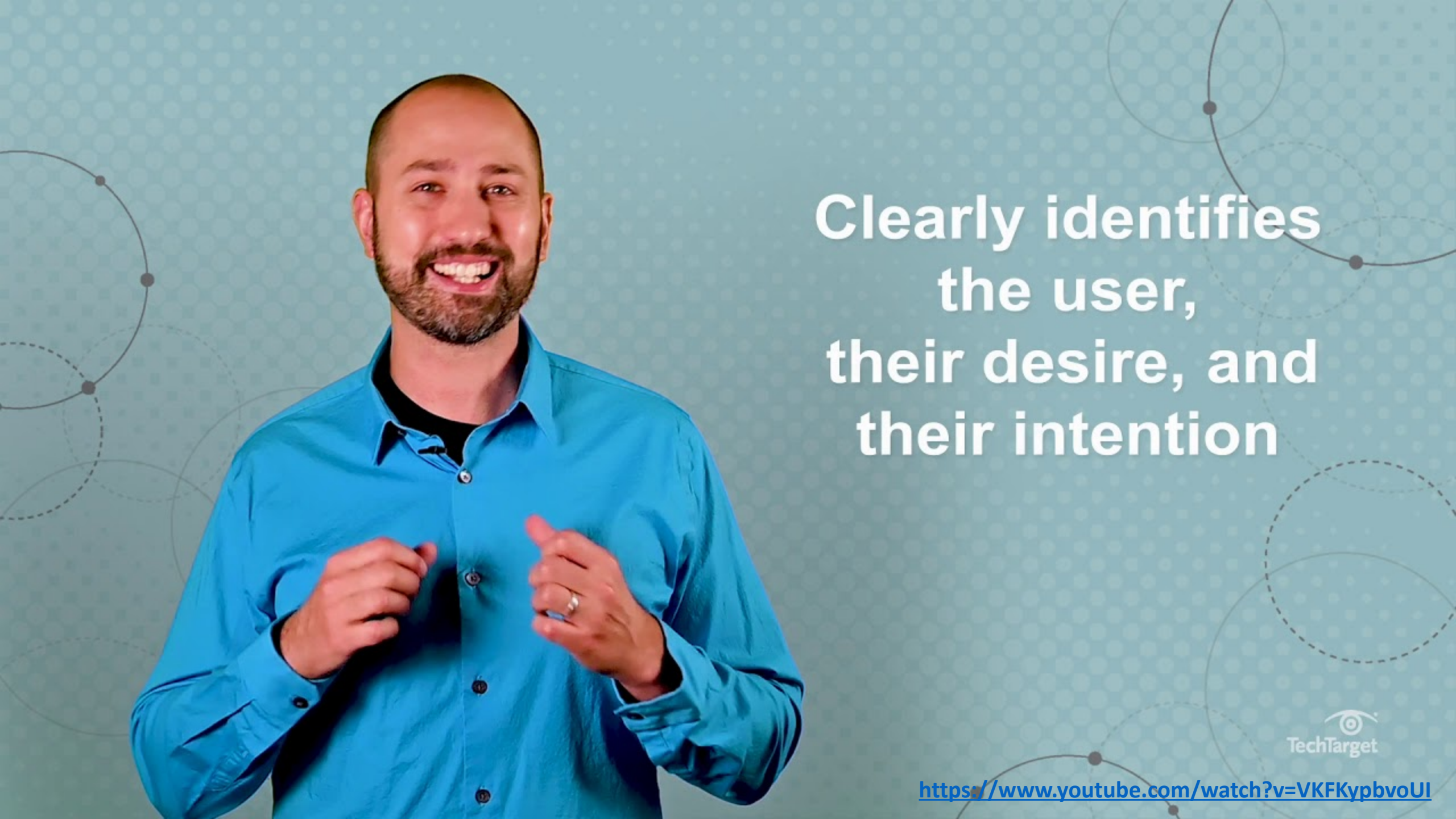
- User Stories / User Story Cards
- INVEST Criteria
- Epics, Story Maps, and Splitting
- User Stories in Scrum

Product Backlog



User stories are seen in product and sprint backlog

User stories comes in 2 forms, direct and indirect. Direct user stories provides value to the customer. Indirect user stories are usually infrstructure set up. Every user story has to be measureable.



https://www.youtube.com/watch?v=VKFKypbvoUI

What is a User Story?

- A user story is a brief, simple description of a feature or functionality from the perspective of an end user

- It typically follows this format:

As a <role>, I want to <perform an action> so that I can <achieve a goal or outcome>

Who

What

A user story has to bring value

Why

For indirect user stories, no need to follow this format. For example, setting up a CICD pipeline.

Examples – Learning Management System (à la eLearn)

User Story #1

As a student, I want to submit my group project files so that I can get them reviewed and graded.

User Story #2

As an instructor, I want to download my section's group project files so that I can review them easily while grading.

What are the **benefits** of describing features as user stories?

*user stories use **non-technical** language to provide context for the whole team*

*user stories put **end users** at the centre of the conversation*

What are the **benefits** of describing features as user stories?

*they convey the **value** of the feature: helpful for prioritisation*

*with the end goal defined, teams can decide how to **collaborate** together to achieve it*

*in short: they facilitate a **conversation** 🗣️*

As a <role>

"As a **customer**, I want to pay by credit card online, so that I can complete my purchase conveniently."



- Specify an abstract role, rather than a specific person or job title
- Helps with generalisation, e.g. thinking about the feature for *any* customer on the system
- Encourages the team to converse about a feature from the perspective of the users who derive value from it

What about features involving **two or more roles**?

I want to <perform an action>

"As a customer, I want to **pay by credit card online**, so that I can complete my purchase conveniently."



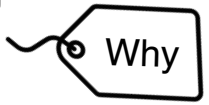
- Specifies **what** the user wants to do

- For example:
  - Want to do something, e.g. see list of credit card transactions
  - Want to get something, e.g. get alert for each credit card transaction
  - Format: {Active verb} {Object}, e.g.:
    - {see list} of {credit card transactions}
    - {get alert} for each {credit card transaction}
  - Object qualifiers, e.g.:
    - can get alert for each credit card transaction of $100 or above

Is this enough to know what needs to be built?

So that I can <achieve a goal or outcome>

"As a customer, I want to pay by credit card online, so that I can **complete my purchase conveniently**."



- Important context – the reason **why** we're building this feature
- (Hopefully) have better conversations among the scrum team and customer to converge on a better solution
- For example – what does "conveniently" mean here?
  - Different credit card providers?
  - Purchase without the need for additional OTPs (faster)?
  - Card details saved?

The goal or outcome should be measurable and specified in the acceptance criteria

User Stories – Rigid Format, Flexible Phrasing

- As a traveler, I want to search for flights by dates so that I can find the most affordable options for my trip
- As a student, I want to access online learning materials offline so that I can study even when I do not have an internet connection
- As a homeowner, I can schedule automated lighting control to enhance security whilst I am away
- As a driver, I can receive real-time traffic updates to avoid congestion
- As a job applicant, I need to upload my resume as an attachment so that I can apply for a job opening
- As a social media user, I need to report abusive accounts so that I can maintain a safe online experience

SMU Classification: Restricted

User Story Card

User Story Card

Title *Good short title*	**Priority** *Order of implementation* High, medium, low based on the value	**Estimate** See Week 4!
User Story *Simple description (no details) of customer/user need* ***(conversation starter)*** *As a <role>, I want to <perform an action> so that I can <achieve a goal or outcome>*		
Acceptance Criteria *List of conditions fulfilled on completion of implementation* Usually in the form of a bullet point. The user story is done when u fulfill all the criteria listed here. If user stories can be broken down, it should be broken down		



- Boundaries of the story
- Use to check if the story ‘done’

User Story Card – Example

Title *Accept Credit Card Payments*	**Priority** *(See Week 4…)*	**Estimate** *(See Week 4…)*
User Story *"As a customer, I want to pay by credit card online, so that I can complete my purchase conveniently."*		
Acceptance Criteria Ensure that the customer: • Can input a valid card number, expiration date, CVV, and cardholder name • Can only pay with Visa, Mastercard, or American Express • Is requested to provide an OTP • Is immediately informed of payment success or failure • Has a secure SSL/TLS connection		

Acceptance Criteria

- Acceptance criteria define the conditions that must **ALL** be met for a user story to be considered complete and acceptable
  - *They should be clear, unambiguous, but not overly specific*
  - *They should be measurable / testable*

What's meant by measurable / testable? Why important?

- They define the 'boundaries' of the user story
  - *What does 'done' look like from the user's perspective?*
  - *What is included within the scope of the story? What is NOT?*

Definition of done and done for user story is 2 different things. Done for user story is the completion of the acceptance criteria

- Used to build consensus within the scrum team
  - *Common understanding on what needs to be built*
  - *In turn, this helps with better planning and estimation* ***(Week 3)***

- Can serve as the basis for test case design **(Week 4)**

User Story

As a customer, i want to buy things online so that it can be delivered to me (too broad)

As a customer, I want to complete an online purchase so that I can receive what I ordered (too broad)

As a customer, I want to add products to a cart so that I can it modify later

Acceptance Criteria

Add up to 10 products, remove products, edit quanity of products, display total price of the cart ...

Acceptance Criteria – **Given-When-Then**

- At times, it can be helpful to elaborate on an acceptance criterion using the **given-when-then** format (aka scenario format)
  - Describes a condition (**given**) under which a triggering event occurs (**when**) resulting in an outcome (**then**)

Checklist-based AC

- Is immediately informed of payment success or failure

Scenario-based ACs
(form of Given-When-Then)

- **Given** a customer has entered valid credit card details
  When they confirm payment
  Then they should receive a payment confirmation message
- **Given** a customer has entered incomplete/invalid card details
  When they confirm payment
  Then a message should advise them to check their card details and payment should not be processed
- …

Checklist- or Scenario-based ACs?

- Checklist-based ACs are concise and faster to write (more agile)
  - Useful when the requirements are simple and do not involve complex user interactions
  - Less can be more! Leaves more detail open to scrum-team negotiation

- Scenario-based ACs provide more context on specific user interactions
  - Useful when the requirements are complex or involve multiple user journeys
  - Or when it's important to understand system behaviour across different conditions

- **Which one to use?** Either or a mix of both, as needed

INVEST Criteria

(i.e. characteristics of good stories)

Writing Good User Stories

Independent

Negotiable

Valuable

Estimable

Small

Testable

A good **user story card** satisfies six attributes.

Bill Wake, *Extreme Programming Explored*, coined the **INVEST** acronym for ease of recall.

Writing Good User Stories

Independent

Ensure User Stories are as independent as possible.

The effort required for each User Story can then be estimated and selected for inclusion in a Sprint **without regard for any other User Stories**.

Valuable

1. As a patient, I can see all **my bills** sorted on date and institution…
2. As a billing executive, I can **create bills** showing different dates for admitting, discharging, billing, invoicing, reminding and paying…

Small

3. As a mobile app user, I can get alerts of nearby restaurants with dining offers
4. As a marketer, I want a location-based service to promote restaurants…

Testable

Interdependent User Stories make estimation and planning more complicated.

Independent

Interdependent – app feature does not work without the server-side feature

How do we build (1) without also building (2) ?

Writing Good User Stories

Independent	A User Story is not cast in stone when written. The "Last Responsible Moment" principle encourages delaying important decisions until the last possible moment without incurring significant risks or negative consequences. Therefore, it is preferable User Story conversations take place as close to coding as possible. **Only when a Sprint starts are the included User Stories fixed until completion of the Sprint.** At all other times, User Stories are **negotiable**, e.g., • Does the User Story fulfil the reason for its existence? • Is it too small and needs to be combined with another User Story? • Is it too big and needs to be broken further? • Are the acceptance criteria sufficient?
Negotiable	
Valuable	
Estimable	
Small	
Testable	

Can you think of ACs that violate the 'negotiable' property?

Writing Good User Stories

Independent **N**egotiable	**"Valuable" is from the viewpoint of the role** for which the User Story is written. The User Story can deliver meaningful value in the form of increased efficiency, improved experience, cost savings, revenue generation, meeting a specific business requirement, etc. An example,
Valuable	As a subscriber, I can be alerted to my unpaid bills, so I can pay punctually and not be charged penalty
Estimable **S**mall **T**estable	This User Story is valuable in that it can potentially save the subscriber penalty charges.

Why do we care about the value of user stories?

Writing Good User Stories

Independent

Negotiable

Valuable

Estimable

Small

Testable

User Stories should be clear enough so that developers can reasonably estimate the required effort.

When might User Stories be unclear:

- **Developers lack domain knowledge** – have User Story conversations to elaborate and specify to the level needed to make an estimate
- **Developers lack technical knowledge**, e.g.,

  As a security officer, I want communications to be protected by TLS, so that sensitive information cannot be intercepted

  Take a time-out to investigate TLS, but strictly within a timebound
- **User Story is too big**, e.g.,

  As a student, I want to communicate with the instructor to schedule a consult, to clarify doubts

  Break it into bite-sizes

  As a student, I want to email the instructor to schedule a consult…

  As a student, I want to Telegram the instructor to schedule a consult…

  As a student, I want to Teams Chat the instructor to schedule a consult…

Is it easier to estimate small work items or large work items?

Writing Good User Stories

Independent

Negotiable

Valuable

Estimable

Small

Testable

Think Goldilocks, not too small, not too big, just the right size for planning:

- If it is too big, it cannot fit within a Sprint, e.g.,

  As a patient, I want so see all information the hospital has of me…

  Break it into bite-sizes

  As a patient, I want to see all my bills…

  As a patient, I want to see all my medical records…
- If it is too small, it might cause inefficiency or lack functional value, e.g.,

  As a patient, I want to see total subsidies…

  As a patient, I want to see all subsidy line items…

  As a patient, I want to see all charges up to two decimal places…

  Club them up to modular size

Which other INVEST properties is this connected to?

Writing Good User Stories

Independent

Negotiable

Valuable

Estimable

Small

Testable

"If the story cannot be tested, how can the developers know when they have finished coding?" – Mike Cohn, *User Stories Applied*

There must be a **clear set of acceptance criteria** for what success looks like and the means to test the criteria:

- The screen must not take too long to appear – this is untestable; how long is long, one second, two seconds, etc.
- The screen must appear within 2 seconds – this is testable

SMU Classification: Restricted

Epics, Story Maps, & Splitting

(aka organising your stories)

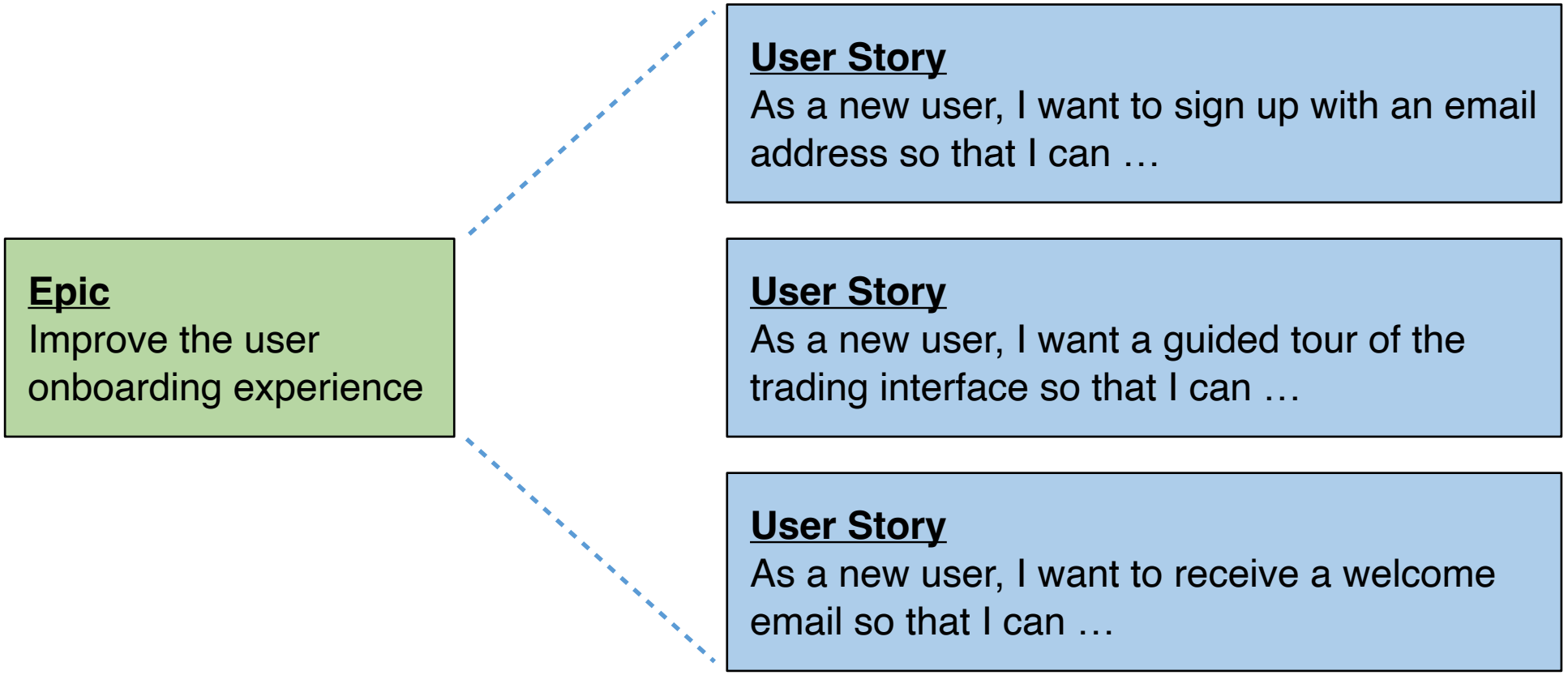
Epics

- Suppose you are implementing a new LMS (e.g. eLearn) for a customer
- The customer has indicated that they want it to have a message board, but that this is a very low-priority feature
- Writing INVEST-able user stories for a message board will require a lot of time. Think of the possible features:
  - Create post, delete post, edit post, notification, swear filter, … 😱
- **Is it worth the effort to write stories for all these features** when your team has many other more pressing features to prioritise?
- **No?** But shouldn't the product backlog list all required functionality … ?

Epics

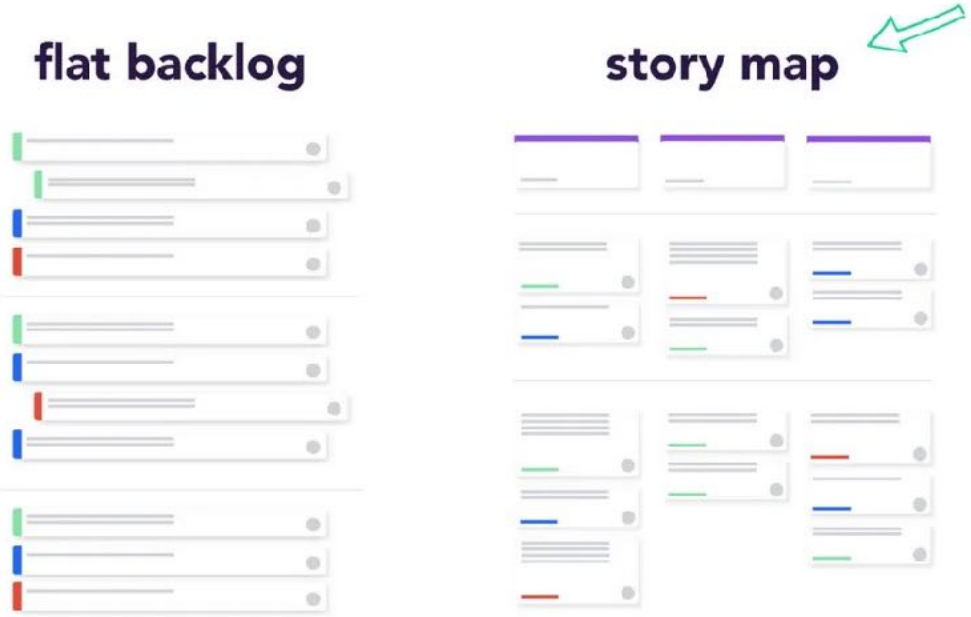
- A solution is to use "epics", which are large stories that represent a broad objective or significant piece of functionality
- Use epics as a placeholder on the product backlog until they are ready to be replaced by (multiple) INVEST-able user stories
- Epics can also be used to help organise your user stories
  - i.e. associate each user story card with a particular overarching epic

Epics – Example



User Story Mapping

Losing The Big Picture



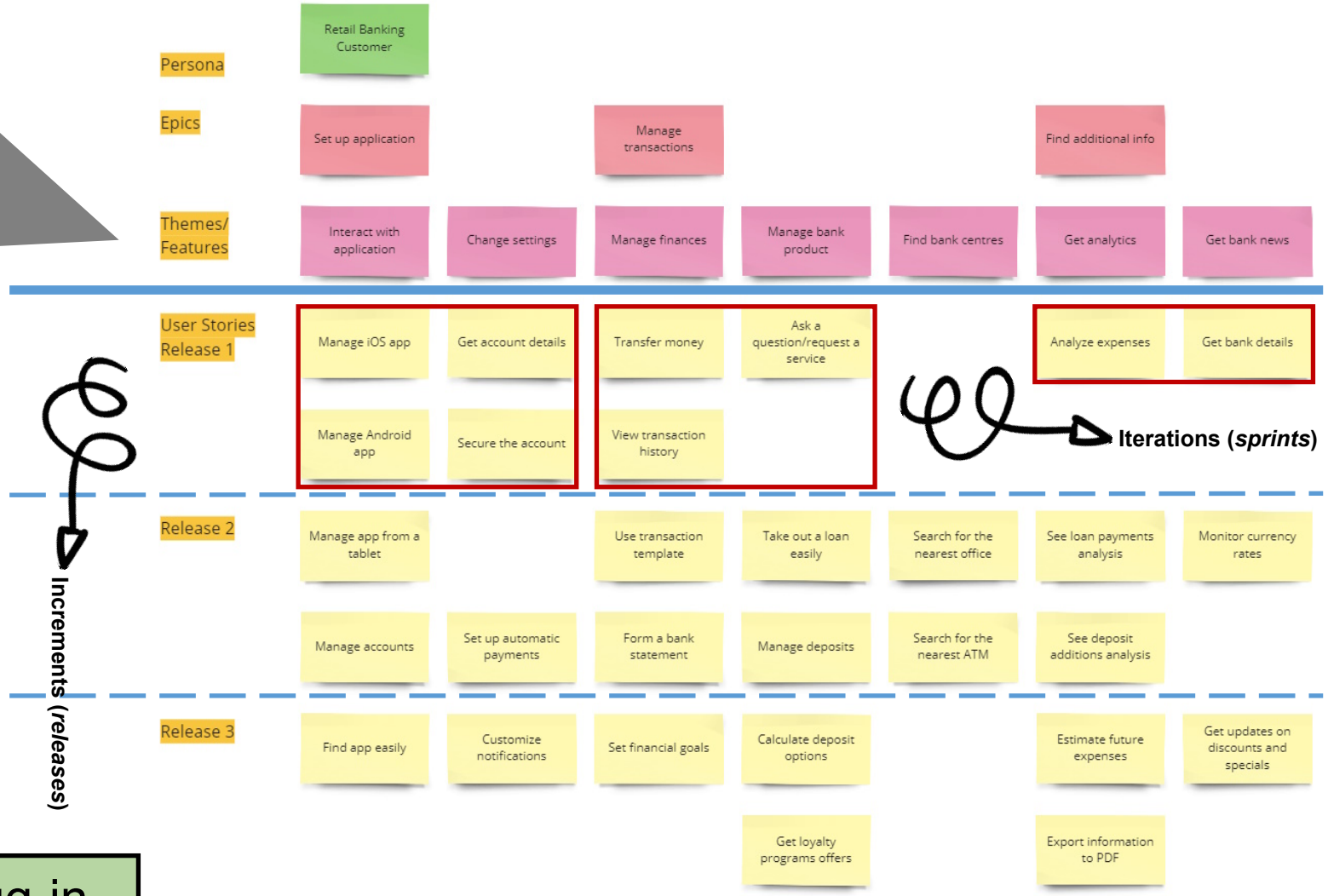
Seeing the Big Picture

‘Story mapping is a technique that provides the big picture that a pile of stories so often misses’

– Martin Fowler

User Story Mapping – Example

- Epics – high-level stories that represent significant functionality (too big to be completed within a Sprint)
- Themes – mid-level stories that represent specific business objectives (too big, so usually split into separate User Stories)
- User Stories – a short and simple description of a single user need



Possible in Jira with a plug-in

Right-sizing your user stories

- Suppose your user story violates the “S” of INVEST
  - i.e. it’s “too large”

- Splitting it into two (or more) smaller, INVEST-able user stories gives you more freedom to plan when they will be implemented

- “Splitting patterns” provide some guidance on how to do this



HOW TO SPLIT A USER STORY



* INVEST - Stories should be:
Independent
Negotiable
Valuable
Estimable
Small
Testable

Source: http://www.richardlawrence.info/splitting-user-stories/

PS: you're not expected to memorise this. Use it as a guide. :-)

Splitting a User Story – Example

Title *Good short title* Show Cinema Location & Movie Schedule	**Priority** *Order of implementation* *(Release/Sprint Planning Use)*	**Estimate** *"Ideal effort" to implement* *(Estimation Use)*
User Story *Simple description (no details) of customer/user want (conversation starter),* As CinemaManager, I want to show the cinema's address **and** movie schedule, so movie goers will visit Too big; "and" is the logical split		
Acceptance Criteria *List of conditions fulfilled on completion of implementation, e.g.,* • Single page showing cinema name, address, nearest MRT and map • List of movies: title, starring, synopsis, show times, ticket charges		

Splitting a User Story – Example

Title *Good short title* Show Cinema Location	**Priority** *Order of implementation* *(Release/Sprint Planning Use)*	**Estimate** *"Ideal effort" to implement* *(Estimation Use)*

User Story
Simple description (no details) of customer/user want (conversation starter), e.g.,

Has
WHO, WHAT , WHY

As CinemaManager, I want to show the cinema's address, so movie goers know how to get here

Acceptance Criteria
List of conditions fulfilled on completion of implementation, e.g.,

And
criteria for "done"

- Single page showing cinema name, address, nearest MRT and map

The movie schedule feature then becomes a different user story card

HOW TO SPLIT A USER STORY



* INVEST - Stories should be:
Independent
Negotiable
Valuable
Estimable
Small
Testable

Source: http://www.richardlawrence.info/splitting-user-stories/

PS: you're not expected to memorise this. Use it as a guide. :-)

User Story

*As a **user**, I can **manage my account** so that …*
—> "manage" implies CRUD. Apply the "Operations" splitting pattern

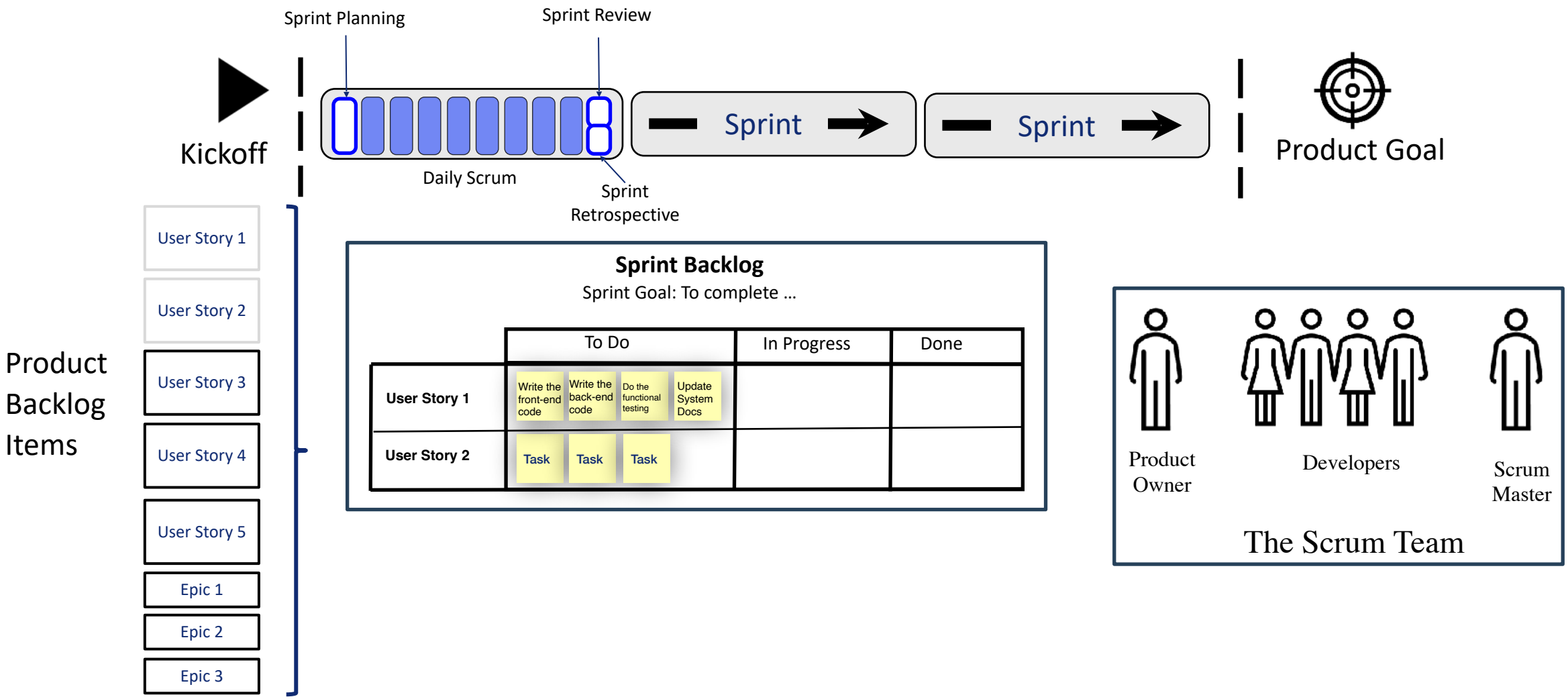
User Story

User Story

User Story

User Stories in Scrum

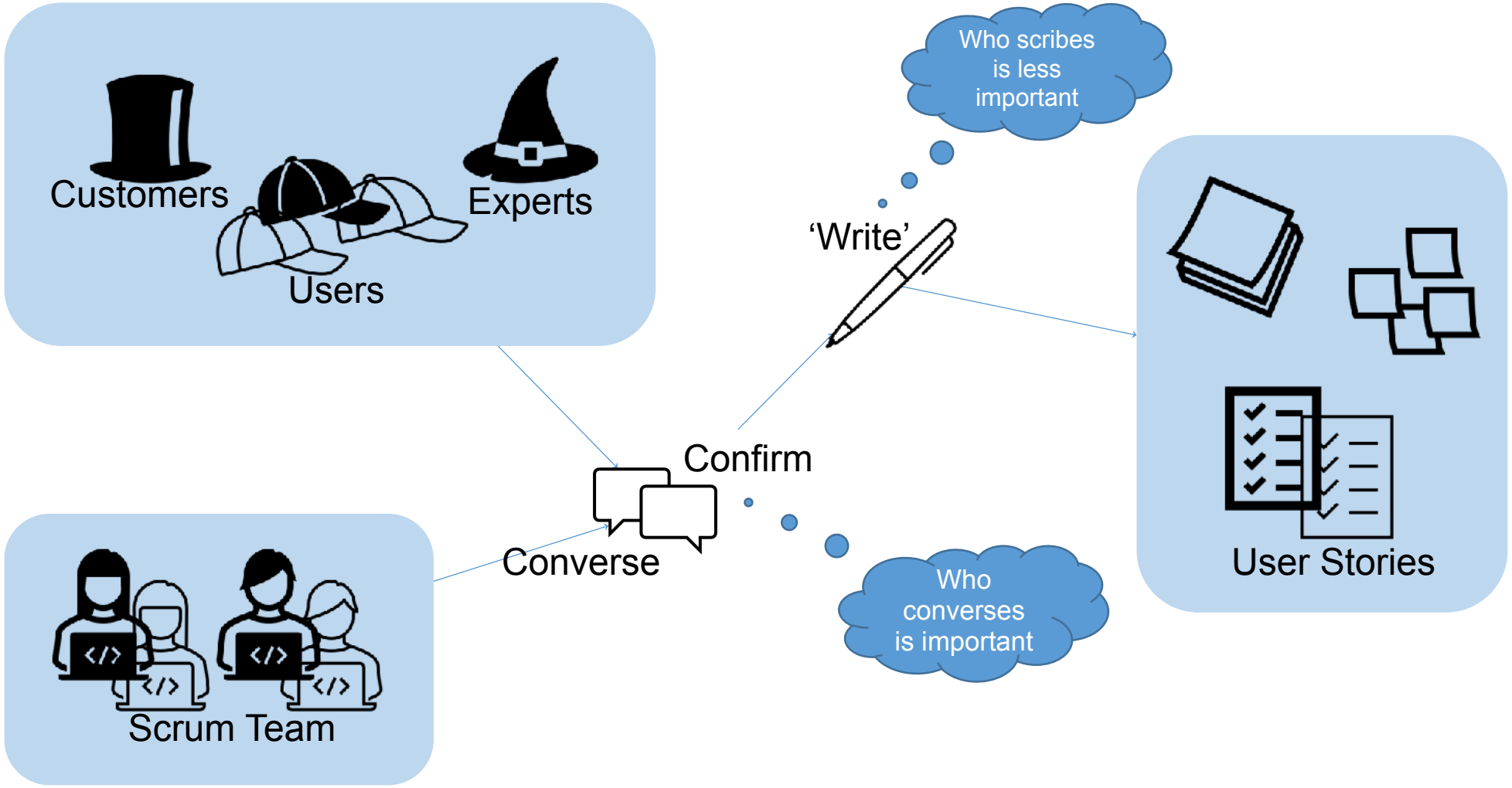
User Stories & Epics are Groomed in the Product Backlog



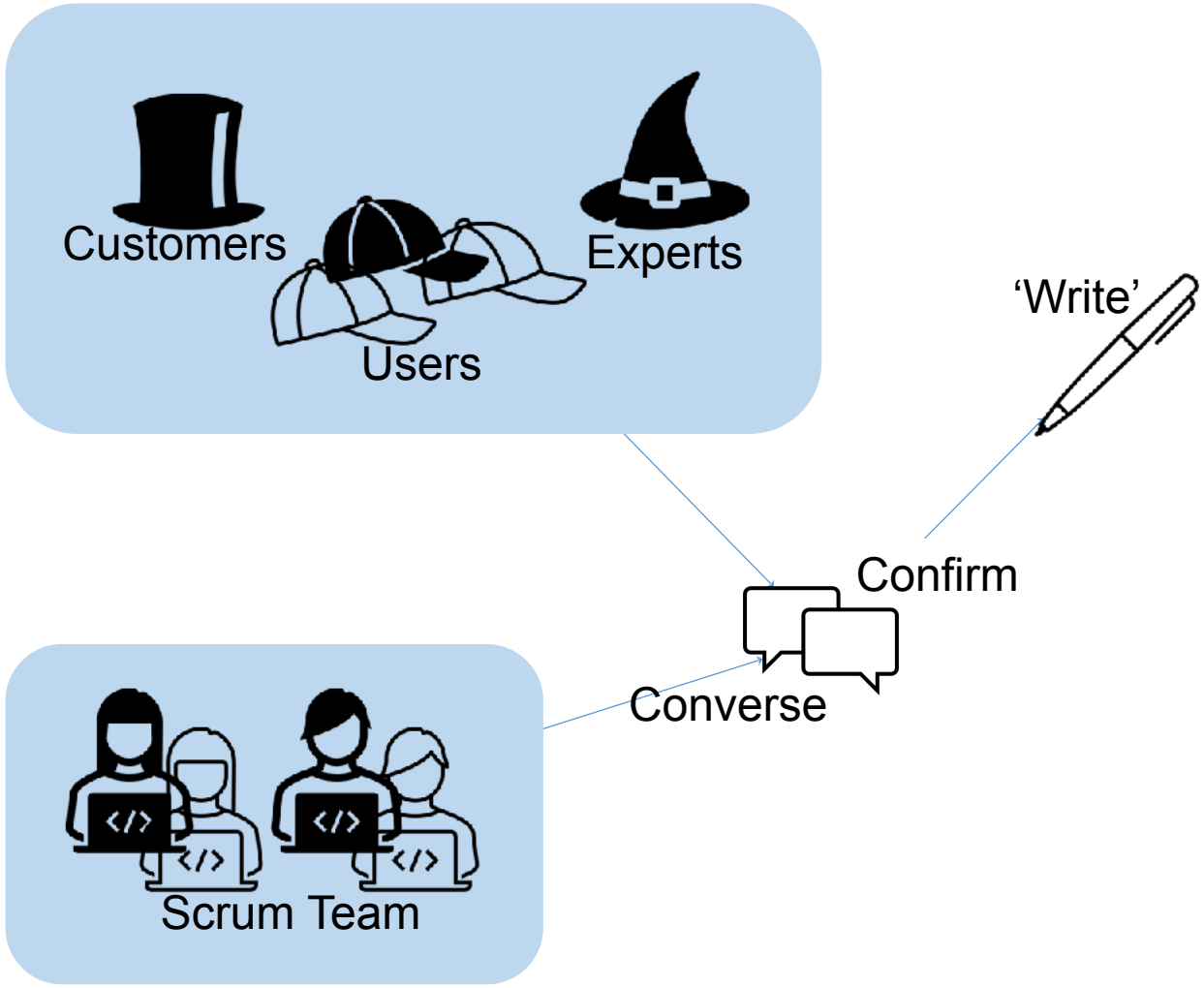
User Stories & Epics are Groomed in the Product Backlog

- The product backlog is a living artefact – it reflects the team's current understanding of what needs to be built
- User stories and epics can be updated, split, re-estimated, or re-prioritised in the product backlog at any time
- Conversations within the team and with the customer are key to this process
- In scrum, which team member has ultimate responsibility for the product backlog?

User Stories & Epics are Groomed in the Product Backlog



User Stories & Epics are Groomed in the Product Backlog



Mike Cohn…

'Software requirements is a communication problem…'

'When the business side dominates, it mandates functionality and dates with little concern that the developers can meet both objectives…'

'When the developers dominate… technical jargon replaces the language of the business… developers lose the opportunity to learn what is needed by listening…'

'If either side dominates these communications, the project loses…'

How ‘Conversations’ Go

Roles

- There is no one ‘User’
- There are many ‘Roles’ each with their own needs
  - ➢ What are these roles?
  - ➢ Name the roles
  - ➢ Group them based on similar pattern of use
  - ➢ Rename intuitively as needed
  - ➢ Use these roles to enumerate user stories

User Stories

- User Stories are not all obvious or of the same size
- They can be apparent at different moments, be of different size and will change with time
- The aim is to capture as many as apparent with the thought that there are more
- As with traditional business analysis, various techniques are useful to trawl for user stories
  - ➢ **Questionnaire** – for a start with large user base
  - ➢ **Observation** – insights from visible/invisible observation and/or user demonstration
  - ➢ **User Interview** – structured, focused, non-leading interview of curated users
  - ➢ **Story-writing Workshop** – whole team lo-fidelity brainstorming, prototyping, role-playing, etc.

How Much to Write?

A good product backlog is…



How Much to Write?

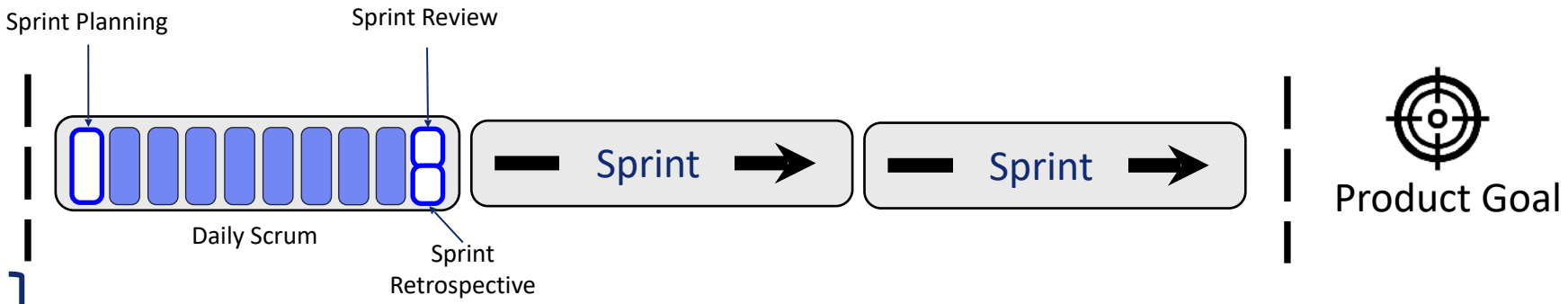
- **Detailed appropriately** – the top priority items are more fine-grained and detailed than the lower priority items, since the former will be worked on sooner than the latter. For example, the top 10% of the backlog may be composed of very small, well-analysed items, and the other 90% much less so

- **Estimated** – the items for the current release need to have estimates, and furthermore, should be considered for re-estimation each Sprint as everyone learns and new information arises. **(Week 3)**

- **Emergent** – in response to learning and variability, the Product Backlog is regularly refined. Each Sprint, items may be added, removed, modified, split, and changed in priority. Thus, the Product Backlog is continuously updated by the Product Owner to reflect changes in the needs of the customer, new ideas or insights, moves by the competition, and technical hurdles that appear.

- **Prioritised** – the items at the top of the Product Backlog are prioritized or ordered in a 1-N order. In general, the highest-priority items should deliver the most bang for your buck: lots of bang (business value) for low buck (cost). Another motivation to increase the priority of an item is to tackle high risks early, before the risks attack you

What about the Sprint Backlog?



Product Backlog Items

- User Story 1
- User Story 2
- User Story 3
- User Story 4
- User Story 5
- Epic 1

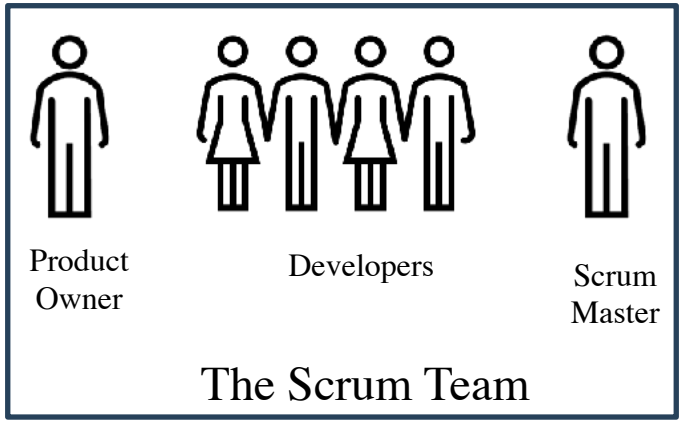
① User Stories can be added, removed or changed

Sprint Backlog

Sprint Goal: To complete …

	To Do	In Progress	Done
User Story 1	Write the front-end code; Write the back-end code; Do the functional testing; Update System Docs		
User Story 2	Task; Task; Task		

② Except in a sprint, where their scope should be **fixed** for the duration

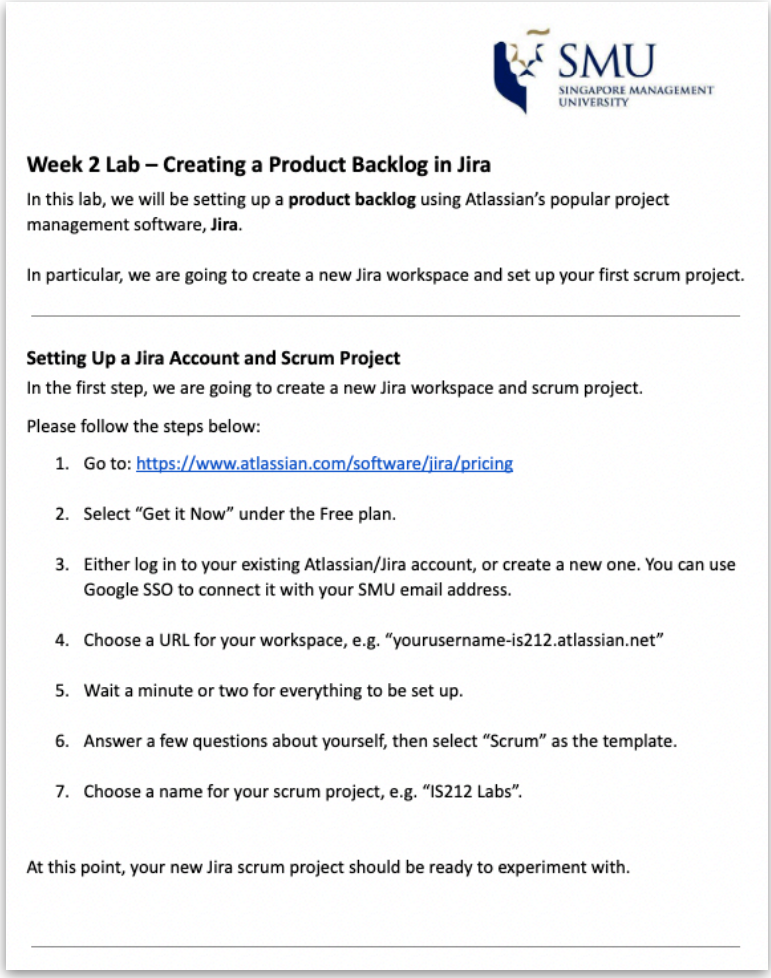


More on sprint planning in **Week 3**!

Lab & Assignment

Lab Exercise – Product Backlog in Jira

- Access the **Week 2 Lab Exercise** in eLearn
- It will show you how to create a product backlog in Jira, a popular project management platform
- This is for illustration only – you can use any project management software you like for the course project



SMU
SINGAPORE MANAGEMENT UNIVERSITY

Week 2 Lab – Creating a Product Backlog in Jira

In this lab, we will be setting up a **product backlog** using Atlassian's popular project management software, **Jira**.

In particular, we are going to create a new Jira workspace and set up your first scrum project.

Setting Up a Jira Account and Scrum Project

In the first step, we are going to create a new Jira workspace and scrum project.

Please follow the steps below:

1. Go to: https://www.atlassian.com/software/jira/pricing
2. Select "Get it Now" under the Free plan.
3. Either log in to your existing Atlassian/Jira account, or create a new one. You can use Google SSO to connect it with your SMU email address.
4. Choose a URL for your workspace, e.g. "yourusername-is212.atlassian.net"
5. Wait a minute or two for everything to be set up.
6. Answer a few questions about yourself, then select "Scrum" as the template.
7. Choose a name for your scrum project, e.g. "IS212 Labs".

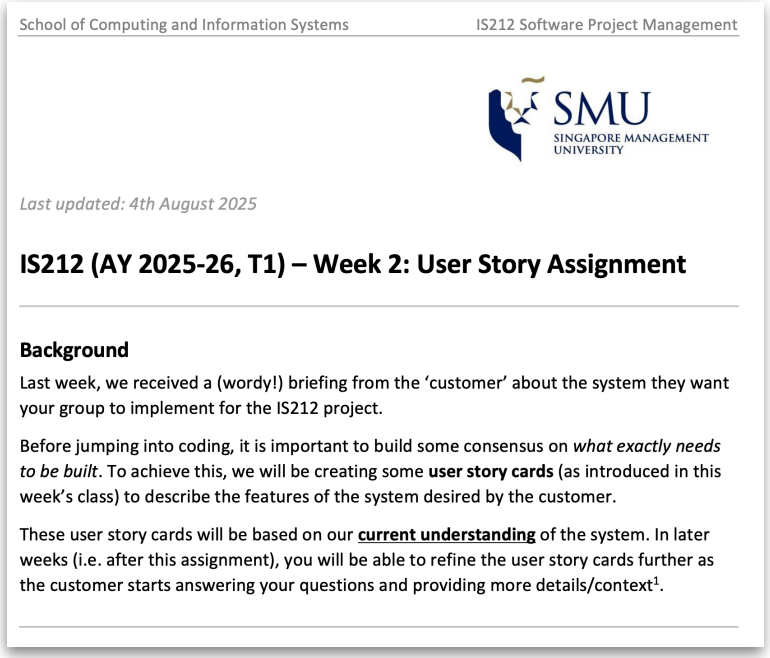
At this point, your new Jira scrum project should be ready to experiment with.

Individual Assignment #1 – User Stories

- **Individual Assignment #1** is on eLearn

- You are tasked to write three INVEST-able user story cards based on the project's customer briefing
  - *"Divide and conquer!"*

- Worth 4% of the grade for IS212



School of Computing and Information Systems — IS212 Software Project Management

SMU SINGAPORE MANAGEMENT UNIVERSITY

Last updated: 4th August 2025

IS212 (AY 2025-26, T1) – Week 2: User Story Assignment

Background

Last week, we received a (wordy!) briefing from the 'customer' about the system they want your group to implement for the IS212 project.

Before jumping into coding, it is important to build some consensus on *what exactly needs to be built*. To achieve this, we will be creating some **user story cards** (as introduced in this week's class) to describe the features of the system desired by the customer.

These user story cards will be based on our **current understanding** of the system. In later weeks (i.e. after this assignment), you will be able to refine the user story cards further as the customer starts answering your questions and providing more details/context[1].

Due 23:59 three days after class
(1 sec late = 50% pen.)

3 user stories that has direct impact

Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet ☺
- **http://smu.sg/is212-reflections-2025**

